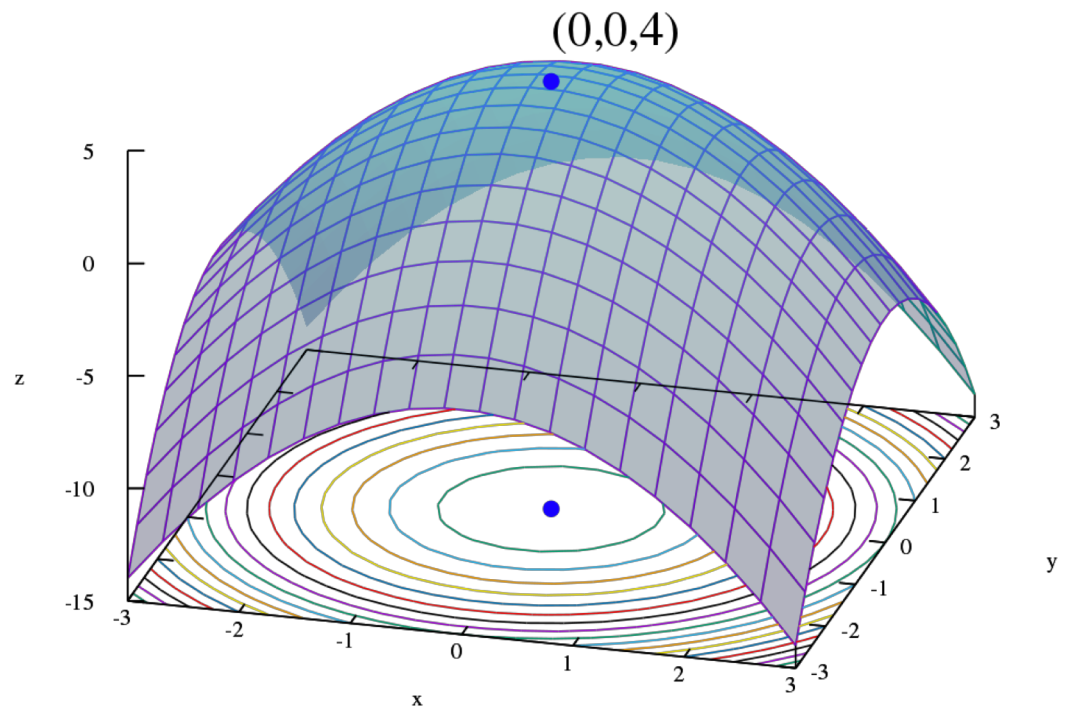
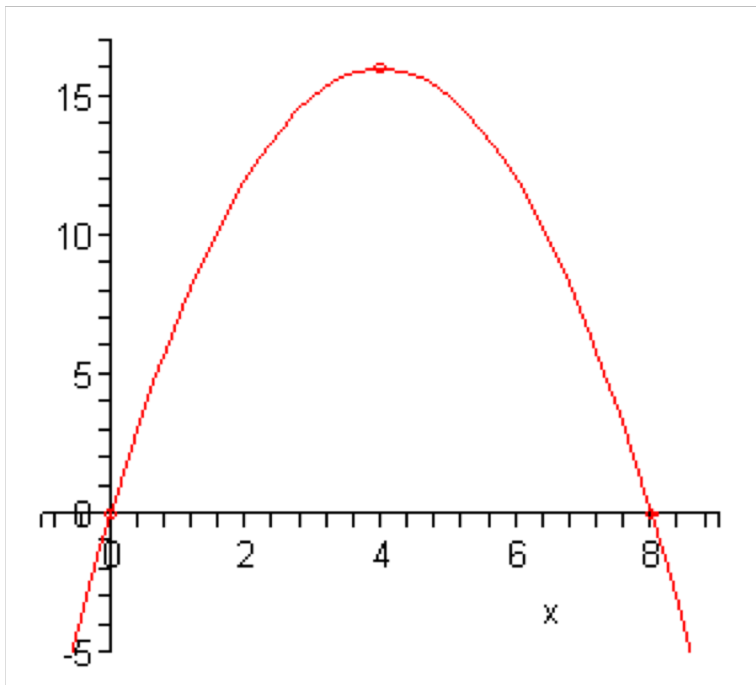
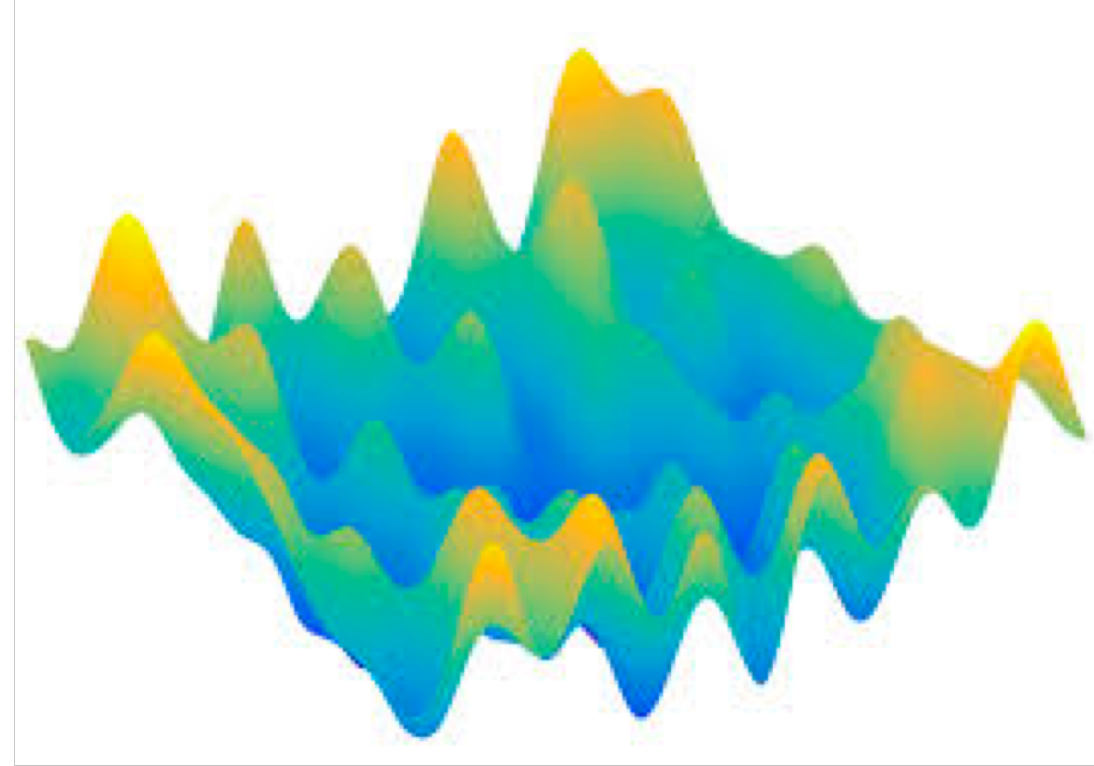


Section 2: Learning

Richard Diehl Martinez, Chuma Kabaghe

Why Learning?





Datapoints

$$\begin{bmatrix} 1.5 \\ 40.0 \\ \vdots \\ 32.0 \\ -4 \end{bmatrix}$$

$\mathbf{x}_i$



Ground Truth
1.0

y_i

Model

$$\begin{bmatrix} 1.5 \\ 40.0 \\ \vdots \\ 32.0 \\ -4 \end{bmatrix}$$

$\mathbf{x}_i$

Model
(parameterized
by $\mathbf{w}$)

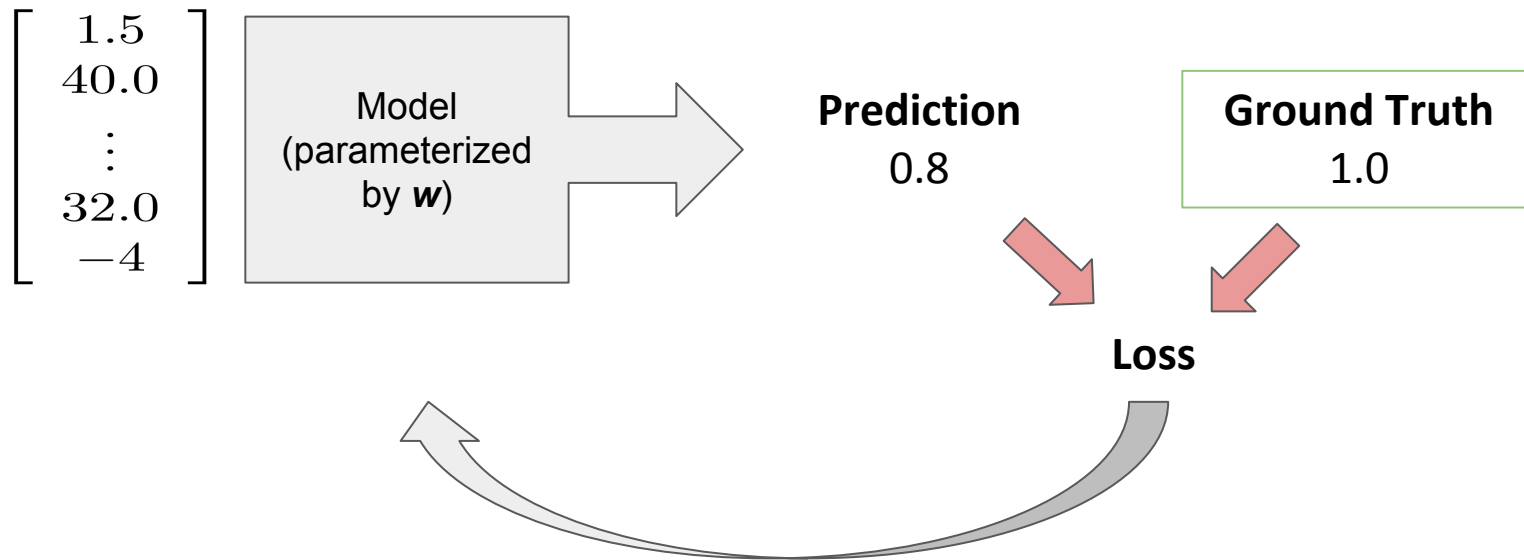
Prediction
0.8

$\hat{y}_i$

Ground Truth
1.0

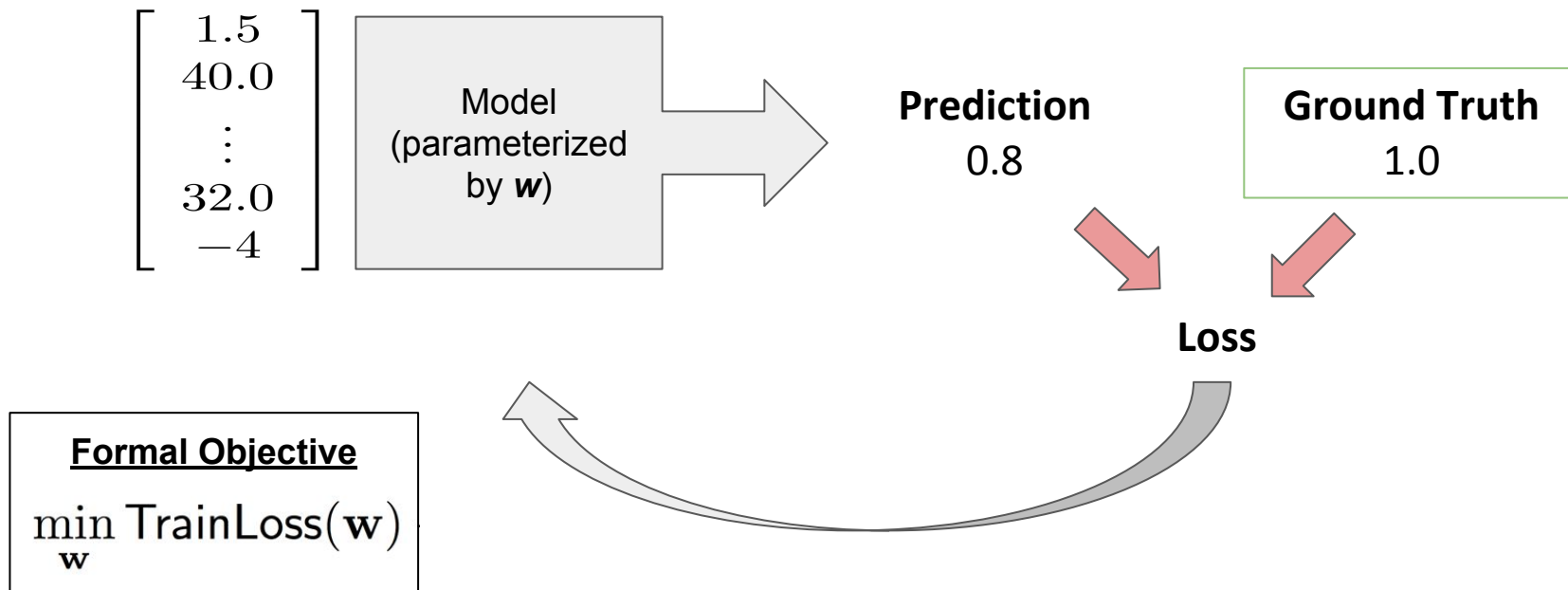
y_i

Loss



Key idea: Use loss to inform updates to weights.

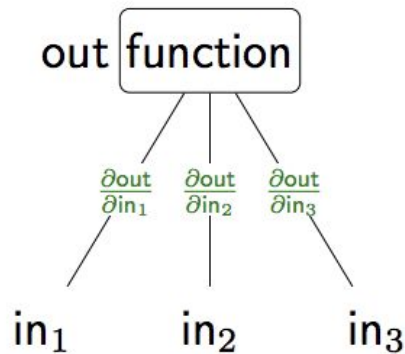
Loss



Key idea: Use loss to inform updates to weights.

Partial Derivatives / Gradients

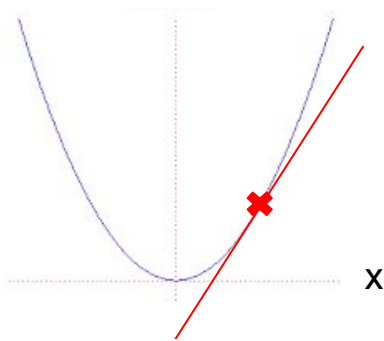
We want to know how each weight affects the training loss.
→ exactly what derivative (gradient in the vector case) tells us!



Partial derivatives (gradients): how much does the output change if an input changes?

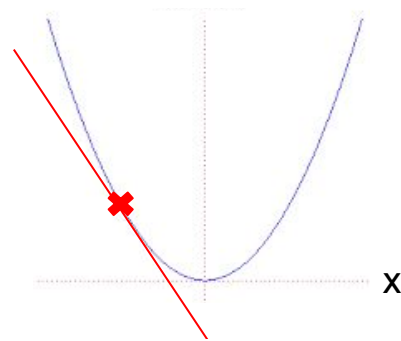
Gradient Descent

We want to know how each weight affects the training loss.
→ exactly what derivative (gradient in the vector case) tells us!



Positive gradient =
decrease x

$$y = x^2$$



Negative gradient =
increase x

Stochastic Gradient Descent



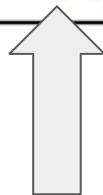
Algorithm: stochastic gradient descent

Initialize $\mathbf{w} = [0, \dots, 0]$

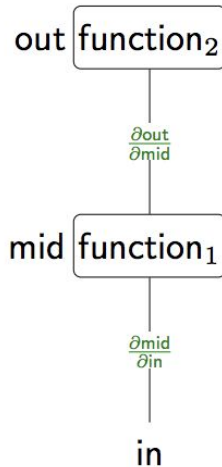
For $t = 1, \dots, T$:

For $(x, y) \in \mathcal{D}_{\text{train}}$:

$\mathbf{w} \leftarrow \mathbf{w} - \eta_t \nabla_{\mathbf{w}} \text{Loss}(x, y, \mathbf{w})$



Backpropagation



Chain rule: $\frac{\partial \text{out}}{\partial \text{in}} = \frac{\partial \text{out}}{\partial \text{mid}} \frac{\partial \text{mid}}{\partial \text{in}}$

Why Backpropagate?

- Don't have to deal with the nastiness of the chain rule with deep neural networks
- Performance optimizations (can hold onto intermediary values, don't have to recompute)
- Translates into a modular framework, so packages like Tensorflow and PyTorch will auto-differentiate for you!

Backpropagation

Backprop: <http://cs231n.github.io/optimization-2/>

Vector, Matrix, and Tensor Derivatives: <http://cs231n.stanford.edu/vecDerivs.pdf>

Nearest neighbors



Algorithm: nearest neighbors

Training: just store $\mathcal{D}_{\text{train}}$

Predictor $f(x')$:

- Find $(x, y) \in \mathcal{D}_{\text{train}}$ where $\|\phi(x) - \phi(x')\|$ is smallest
- Return y

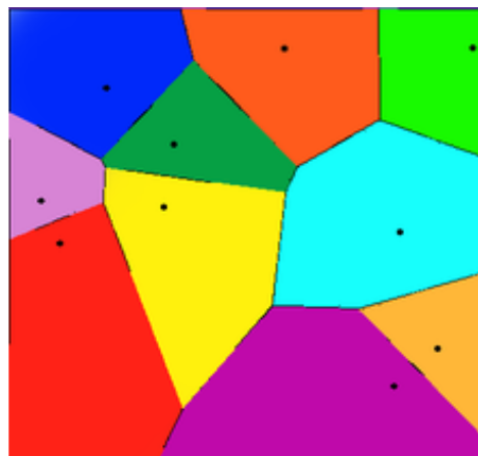


Key idea: similarity

Similar examples tend to have similar outputs.

Expressivity of nearest neighbors

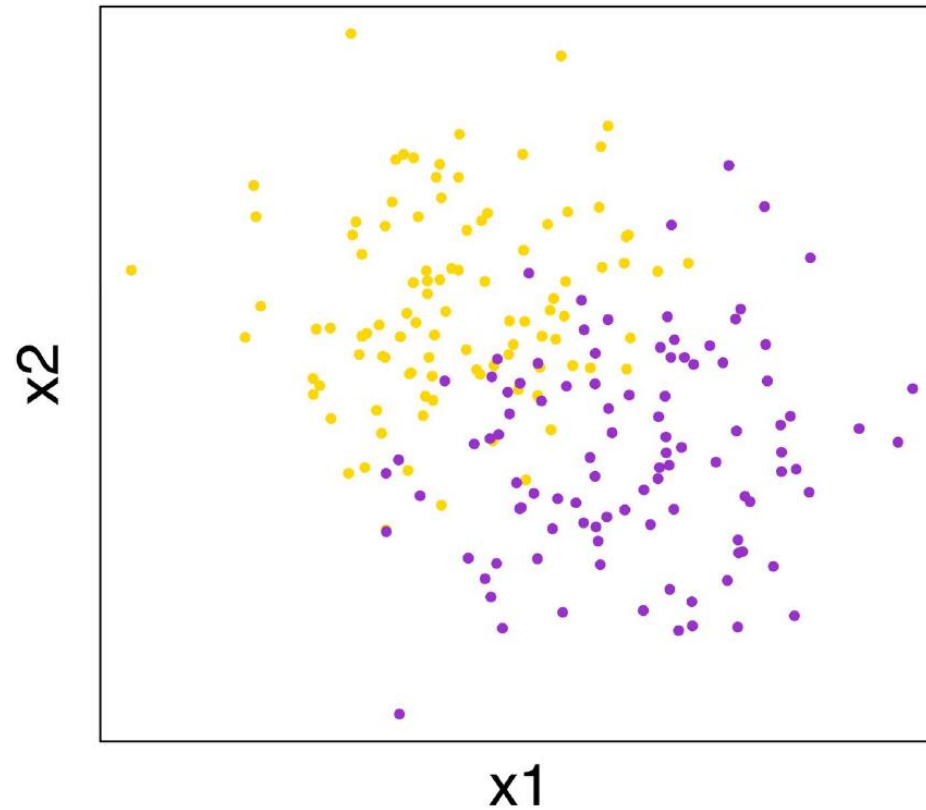
Decision boundary: based on Voronoi diagram



- Much more expressive than quadratic features
- **Non-parametric**: the hypothesis class adapts to number of examples
- Simple and powerful, but kind of brute force

k -nearest neighbors

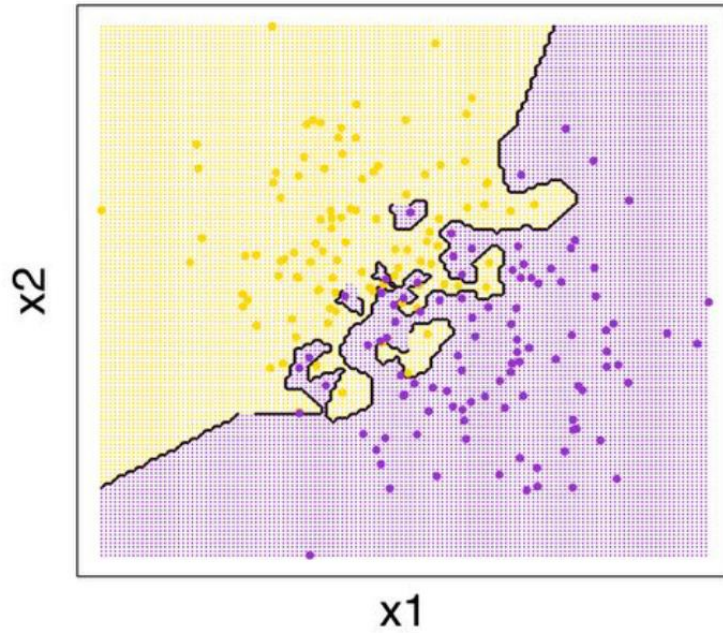
Binary kNN Classification Training Set



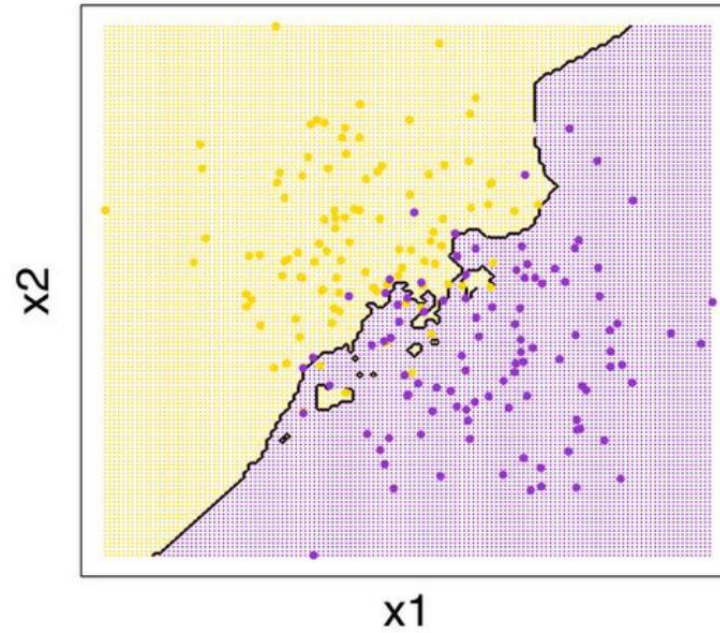
k -nearest neighbors

Effect of k :

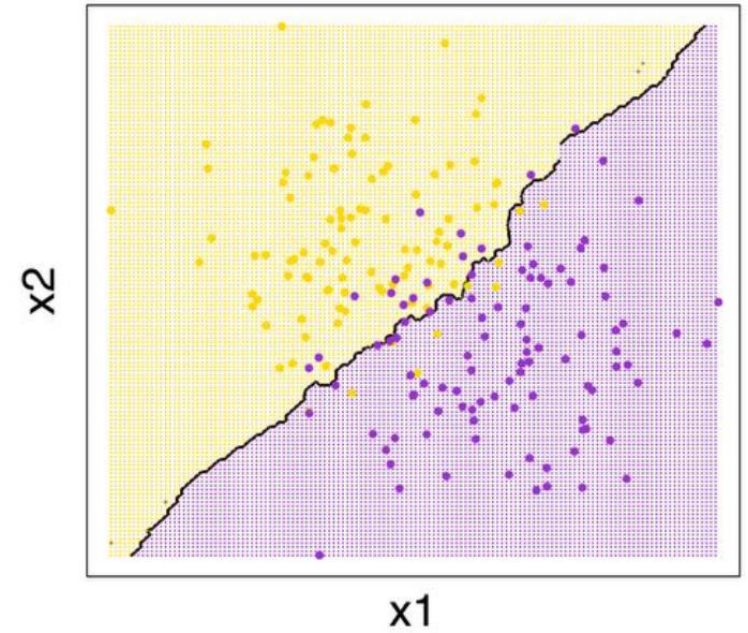
Binary kNN Classification ($k=1$)



Binary kNN Classification ($k=5$)



Binary kNN Classification ($k=25$)



[Switch to ipython notebook]

k -fold cross-validation

